

安全漏洞审计报告 · 第7轮

用户信息管理平台

SSRF 服务端请求伪造漏洞审计报告

User Management Platform — Server-Side Request Forgery Assessment

目标系统：用户信息管理平台 (Flask)

报告日期：2026-07-15

漏洞等级：严重 × 2

修复状态：✅ 全部已修复

目录

1. 报告概述
2. 漏洞详情
3. 攻击演示
4. 修复方案
5. 修复前后对照
6. 修复验证
7. 全部七轮汇总

1. 报告概述

1.1 本轮背景

系统新增了**URL** 抓取功能（/fetch-url），允许用户输入 URL 让服务器抓取内容。由于没有对 URL 做任何限制，攻击者可利用 file:// 读取服务器任意文件，或通过内网地址扫描和攻击内部服务。

1.2 什么是 SSRF？

SSRF（Server-Side Request Forgery） 服务器端请求伪造。攻击者让服务器代替自己向内网或其他受限网络发起请求。可用来扫描内网端口、攻击内部服务、读取本地文件（结合 file:// 协议）。

2. 漏洞详情

2

● 严重漏洞

2/2

✓ 已修复

VULN-23 : SSRF 攻击内网服务

● 严重

位置 : /fetch-url — POST 路由

描述 : 未对 URL 的目标 IP 做任何限制, 攻击者能让服务器访问内网地址 (127.0.0.1、192.168.x.x、10.x.x.x), 用于扫描内网端口或攻击内部服务。

攻击场景 : url=http://127.0.0.1:5000/ → 访问本机 Flask 服务 ; url=http://192.168.78.130:3306 → 扫描内网 MySQL 端口。

VULN-24 : file:// 协议读取任意文件

● 严重

位置 : /fetch-url — POST 路由

描述 : 未限制协议类型, 攻击者可使用 file:// 协议读取服务器上的任意本地文件, 包括源码、配置文件和系统敏感文件。

攻击场景 : url=file:///etc/passwd → 读取系统用户列表 ; url=file:///etc/shadow → 读取密码哈希。

3. 攻击演示

3.1 file:// 读取系统密码文件

攻击请求

```
POST /fetch-url url=file:///etc/passwd
```

攻击结果

成功读取系统用户列表：

```
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

3.2 读取密码哈希

攻击请求

```
POST /fetch-url url=file:///etc/shadow
```

攻击结果

成功读取到密码哈希：

```
root:$y$j9T$pl0EafxSTnm.o4cBGWzZL1$...
```

3.3 SSRF 扫描内网端口

攻击请求

```
POST /fetch-url url=http://127.0.0.1:5000/
```

攻击结果

可探测到内网服务：

端口 22 (SSH)  | 端口 80 (HTTP)  | 端口 3306 (MySQL) 

4. 修复方案

4.1 限制协议

```
parsed = urllib.parse.urlparse(target_url)
if parsed.scheme not in ("http", "https"):
    fetch_result = "不支持的协议，仅允许 http/https"
```

4.2 阻止内网地址

```
ip = socket.gethostbyname(hostname) # 解析域名到 IP
ip_obj = ipaddress.ip_address(ip)
if ip_obj.is_private or ip_obj.is_loopback or ip_obj.is_link_local:
    fetch_result = "不允许访问内网地址"
```

5. 修复前后对照

✖ 修复前

```
target_url = request.form.get("url")
urllib.request.urlopen(target_url)
```

- file:///etc/passwd ✔ 可读取
- http://127.0.0.1:5000/ ✔ 可访问
- 任何协议、任何 IP 都放行

✔ 修复后

```
if scheme not in ("http","https"): reject
ip = socket.gethostbyname(hostname)
if is_private: reject
```

- file:///etc/passwd ❌ 被拦截
- http://127.0.0.1:5000/ ❌ 被拦截
- 外网 http/https 正常使用

6. 修复验证

#	测试	结果
1	file:///etc/passwd → 被拦截	✓ 通过
2	http://127.0.0.1:5000/ → 被拦截	✓ 通过
3	http://192.168.78.130:5000/ → 被拦截	✓ 通过
4	http://httpbin.org/get → 正常	✓ 通过
5	https://example.com → 正常	✓ 通过

7. 全部七轮汇总

轮次	审计内容	漏洞数	严重	高危	中危	状态
第 1 轮	密码安全	8	4	-	4	✓
第 2 轮	SQL 注入	2	2	-	-	✓
第 3 轮	文件上传	1	1	-	-	✓
第 4 轮	业务逻辑	7	2	2	3	✓
第 5 轮	文件包含	2	2	-	-	✓
第 6 轮	CSRF	2	1	1	-	✓
第 7 轮	SSRF	2	2	-	-	✓
合计		24	14	3	7	全部已修复

安全漏洞审计报告 · 第7轮 — SSRF 安全 | 报告日期：2026-07-15

项目地址：<https://github.com/sencly-book/user-management>